Applying the SparkUI

In this train, we'll learn how to use the SparkUI to monitor and optimise Spark applications. We'll focus on interpreting the SparkUI's metrics and visualizations to identify performance bottlenecks and improve job execution.

Learning Objectives

By the end of this train, you should be able to:

- Apply the SparkUI.

Apply the SparkUI

In using Spark, you may have come across the Spark UI, especially if things have started to go wrong. The Spark UI is a graphical user interface into the workings of the Spark application. You can have a look at the jobs, stages, and tasks, some storage elements, the environment, gain information on the executors, memory usage, and even have a look at the SparkSQL queries that have been executed.

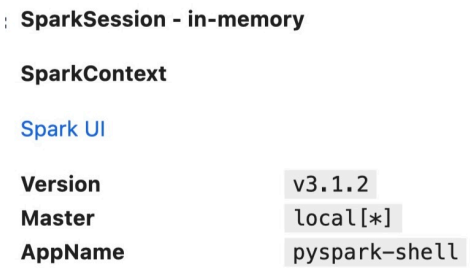
Let's access the Spark UI by calling the SparkSession and following the Spark UI hyperlink.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()

spark
```

Output:



When you open up the Spark UI, it should land you on a page as shown in Figure 1.

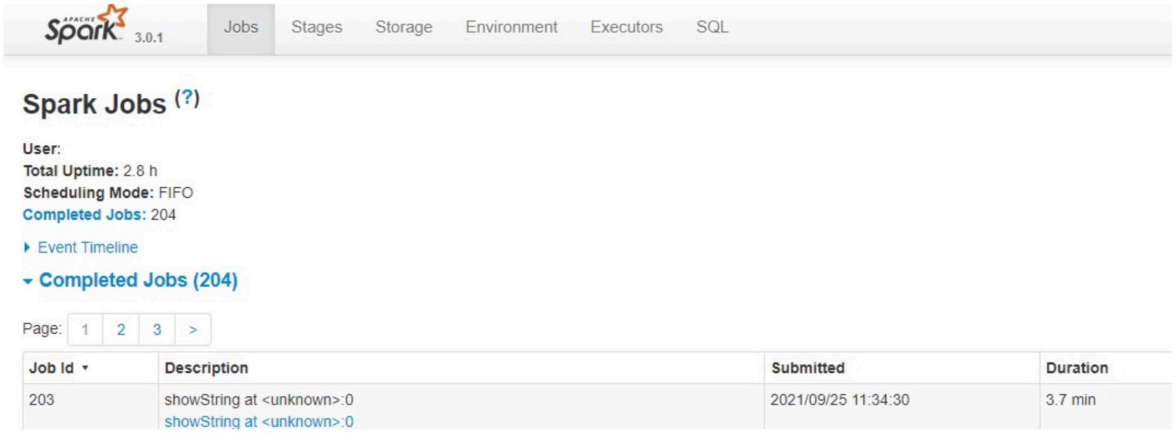


Figure 1: The Spark UI Landing Page.

From this, you can locate the jobs, stages, storage, environment, executors, and sql tabs. We will discuss each in a bit more detail below.

Jobs and Stages

Remember that a Spark application is broken down into jobs, stages, and tasks. The Spark UI allows you access to the application through the Jobs and Stages tabs. You can open each of these tabs, and drill down to an increasing level of detail – down to the task level – and gain insight into various metrics for each task, such as memory usage, disk I/O, execution time, and more. The Jobs tab provides a timeline for the execution of a specific job. It shows the time the executor was attached to the cluster (this will be when you started Spark if you are running anything on a local Spark instance). You can also see how long each stage in the job took by inspecting the duration field in the table. This is a crucial place to have a look when you are optimizing your applications, since a single long-running stage may indicate inefficiencies that can be optimized.

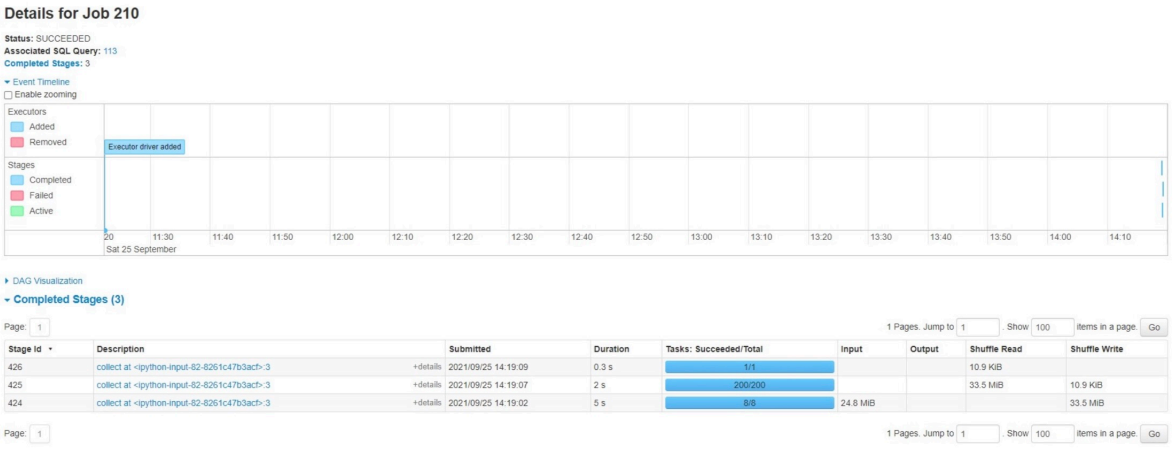


Figure 2: An example of an open job in the Spark UI.

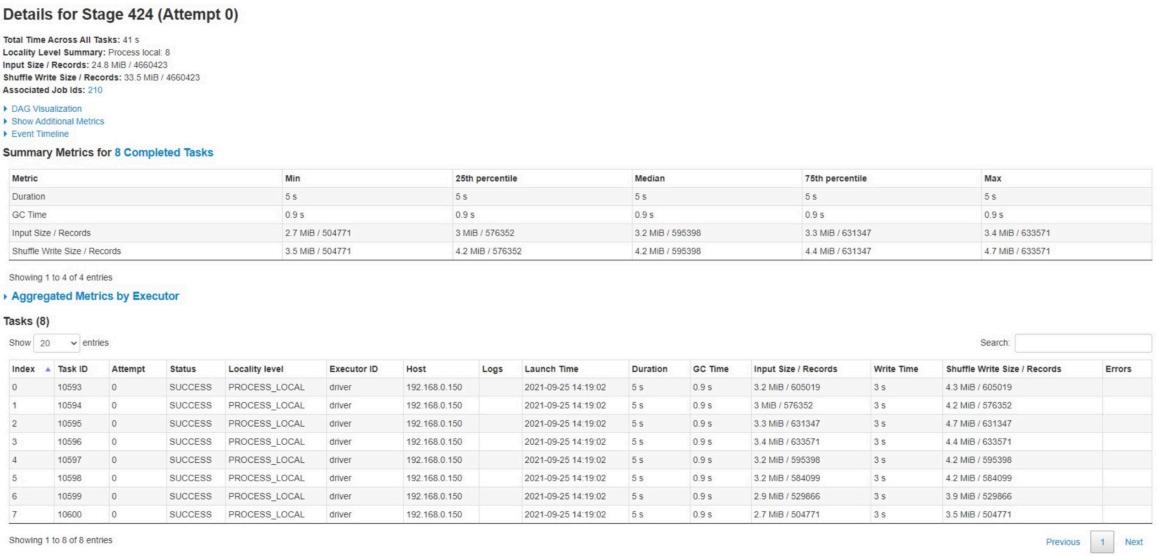
As an example, we open up the third stage of the above job to investigate why it took the longest of all three stages. Similar to the Jobs tab, the Stages tab provides detailed information into the metrics of all stages run within the Spark instance. Again, it is possible to inspect a specific stage here. Some of the available metrics are:

the average duration of each task;

time spent on garbage collection; and

the number of shuffle bytes.

Irregular values in each of these indicate a specific issue. If the variance in task duration is high, it may indicate that your partitions are skewed. If the time spent on garbage collection is high, it may indicate that your workers do not have enough memory allocated, and high I/O may indicate that data needs to be read from external executors frequently.



Details for Stage 424 (Attempt 0)

Total Time Across All Tasks: 41 s
Locality Level Summary: Process local: 8
Input Size / Records: 24.8 MiB / 4660423
Shuffle Write Size / Records: 33.5 MiB / 4660423
Associated Job Ids: 210

▸ DAG Visualization
▸ Show Additional Metrics
▸ Event Timeline

Summary Metrics for 8 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	5 s	5 s	5 s	5 s	5 s
GC Time	0.9 s	0.9 s	0.9 s	0.9 s	0.9 s
Input Size / Records	2.7 MiB / 504771	3 MiB / 576352	3.2 MiB / 595398	3.3 MiB / 631347	3.4 MiB / 633571
Shuffle Write Size / Records	3.5 MiB / 504771	4.2 MiB / 576352	4.2 MiB / 595398	4.4 MiB / 631347	4.7 MiB / 633571

Showing 1 to 4 of 4 entries

▸ Aggregated Metrics by Executor

Tasks (8)

Show 20 entries Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Input Size / Records	Write Time	Shuffle Write Size / Records	Errors
0	10593	0	SUCCESS	PROCESS_LOCAL	driver	192.168.0.150		2021-09-25 14:19:02	5 s	0.9 s	3.2 MiB / 605019	3 s	4.3 MiB / 605019	
1	10594	0	SUCCESS	PROCESS_LOCAL	driver	192.168.0.150		2021-09-25 14:19:02	5 s	0.9 s	3 MiB / 576352	3 s	4.2 MiB / 576352	
2	10595	0	SUCCESS	PROCESS_LOCAL	driver	192.168.0.150		2021-09-25 14:19:02	5 s	0.9 s	3.3 MiB / 631347	3 s	4.7 MiB / 631347	
3	10596	0	SUCCESS	PROCESS_LOCAL	driver	192.168.0.150		2021-09-25 14:19:02	5 s	0.9 s	3.4 MiB / 633571	3 s	4.4 MiB / 633571	
4	10597	0	SUCCESS	PROCESS_LOCAL	driver	192.168.0.150		2021-09-25 14:19:02	5 s	0.9 s	3.2 MiB / 595398	3 s	4.2 MiB / 595398	
5	10598	0	SUCCESS	PROCESS_LOCAL	driver	192.168.0.150		2021-09-25 14:19:02	5 s	0.9 s	3.2 MiB / 584099	3 s	4.2 MiB / 584099	
6	10599	0	SUCCESS	PROCESS_LOCAL	driver	192.168.0.150		2021-09-25 14:19:02	5 s	0.9 s	2.9 MiB / 529866	3 s	3.9 MiB / 529866	
7	10600	0	SUCCESS	PROCESS_LOCAL	driver	192.168.0.150		2021-09-25 14:19:02	5 s	0.9 s	2.7 MiB / 504771	3 s	3.5 MiB / 504771	

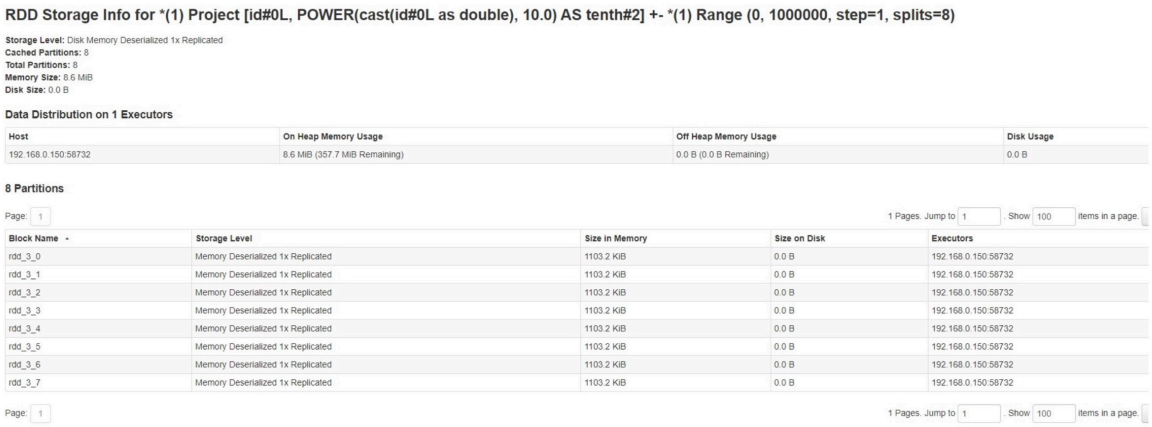
Showing 1 to 8 of 8 entries Previous 1 Next

Figure 3: An example of the Stages tab in the Spark UI

Storage

The Storage tab in the Spark UI provides information on tables or DataFrame caches by the application as a result of `cache()` or `persist()`.

For example, we can have a look at one of the tables that we have cached above.



RDD Storage Info for *(1) Project [id#0L, POWER(cast(id#0L as double), 10.0) AS tenth#2] +- *(1) Range (0, 1000000, step=1, splits=8)

Storage Level: Disk Memory Deserialized 1x Replicated
Cached Partitions: 8
Total Partitions: 8
Memory Size: 8.6 MiB
Disk Size: 0.0 B

Data Distribution on 1 Executors

Host	On Heap Memory Usage	Off Heap Memory Usage	Disk Usage
192.168.0.150:58732	8.6 MiB (357.7 MiB Remaining)	0.0 B (0.0 B Remaining)	0.0 B

8 Partitions

Page: 1 — 1 Pages. Jump to 1. Show 100 items in a page.

Block Name	Storage Level	Size in Memory	Size on Disk	Executors
rdd_3_0	Memory Deserialized 1x Replicated	1103.2 KiB	0.0 B	192.168.0.150:58732
rdd_3_1	Memory Deserialized 1x Replicated	1103.2 KiB	0.0 B	192.168.0.150:58732
rdd_3_2	Memory Deserialized 1x Replicated	1103.2 KiB	0.0 B	192.168.0.150:58732
rdd_3_3	Memory Deserialized 1x Replicated	1103.2 KiB	0.0 B	192.168.0.150:58732
rdd_3_4	Memory Deserialized 1x Replicated	1103.2 KiB	0.0 B	192.168.0.150:58732
rdd_3_5	Memory Deserialized 1x Replicated	1103.2 KiB	0.0 B	192.168.0.150:58732
rdd_3_6	Memory Deserialized 1x Replicated	1103.2 KiB	0.0 B	192.168.0.150:58732
rdd_3_7	Memory Deserialized 1x Replicated	1103.2 KiB	0.0 B	192.168.0.150:58732

Page: 1 — 1 Pages. Jump to 1. Show 100 items in a page.

Figure 4: An example of the cached table. Here we can see a split into eight partitions, each stored in a deserialised format in memory.

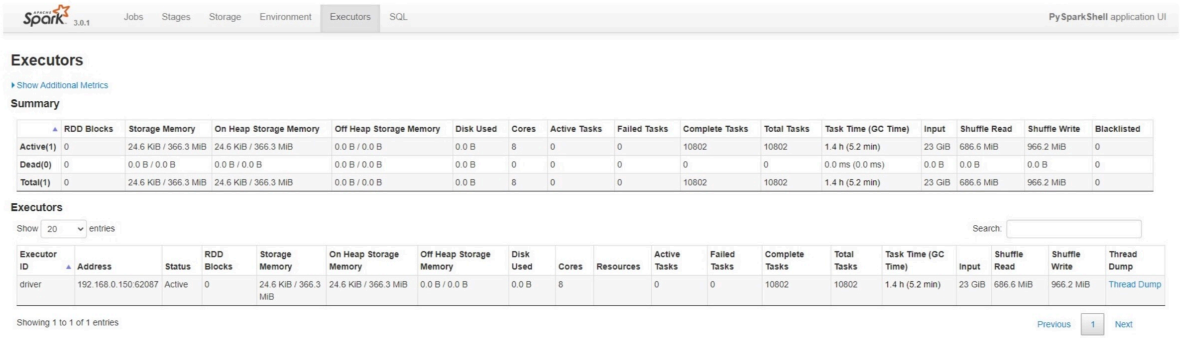
Environment

As mentioned above, it is important to properly configure your Spark application. The Environment tab provides more information on the Spark application and the environment within which it is set up. It provides detail into the Java and Scala versions on which Spark is built, Spark, Hadoop, and System properties. Of these, Spark properties are the most important, providing insight into all of the Spark configurations.

img src="https://raw.githubusercontent.com/Explore-AI/Pictures/master/alx-courses/de/Optimizing_Spark/The_Spark_UI_Environment_tab.jpeg"> *Figure 5: The Spark UI Environment tab.*

Executors

The Executors tab provides more information on the executors available to the application. This is a relatively straightforward tab, as it provides more detail on the basic statistics for all executors, including resource usage, time in garbage collection, completed, failed, and active tasks, as well as the shuffled data, read and written.



Spark 3.0.1 — Jobs | Stages | Storage | Environment | Executors | SQL — PySparkShell application UI

Executors

Show Additional Metrics

Summary

	RDD Blocks	Storage Memory	On Heap Storage Memory	Off Heap Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Blacklisted
Active(1)	0	24.6 KiB / 366.3 MiB	24.6 KiB / 366.3 MiB	0.0 B / 0.0 B	0.0 B	8	0	0	10802	10802	1.4 h (5.2 min)	23 GiB	686.6 MiB	966.2 MiB	0
Dead(0)	0	0.0 B / 0.0 B	0.0 B / 0.0 B	0.0 B / 0.0 B	0.0 B	0	0	0	0	0	0.0 ms (0.0 ms)	0.0 B	0.0 B	0.0 B	0
Total(1)	0	24.6 KiB / 366.3 MiB	24.6 KiB / 366.3 MiB	0.0 B / 0.0 B	0.0 B	8	0	0	10802	10802	1.4 h (5.2 min)	23 GiB	686.6 MiB	966.2 MiB	0

Executors

Show 20 entries — Search:

Executor ID	Address	Status	RDD Blocks	Storage Memory	On Heap Storage Memory	Off Heap Storage Memory	Disk Used	Cores	Resources	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Thread Dump
driver	192.168.0.150:62087	Active	0	24.6 KiB / 366.3 MiB	24.6 KiB / 366.3 MiB	0.0 B / 0.0 B	0.0 B	8		0	0	10802	10802	1.4 h (5.2 min)	23 GiB	686.6 MiB	966.2 MiB	Thread Dump

Showing 1 to 1 of 1 entries — Previous 1 Next

Figure 6: An example of the Executors tab in the Spark UI.

SQL

All queries executed using the SparkSQL interface are first converted back to SparkSQL (even if you write PySpark code). This tab provides you with insight into all SQL queries that ran within the Spark application. It can also give insight into the specific number of rows that were selected, shuffle data amounts, file sizes, execution times, or other useful statistics.

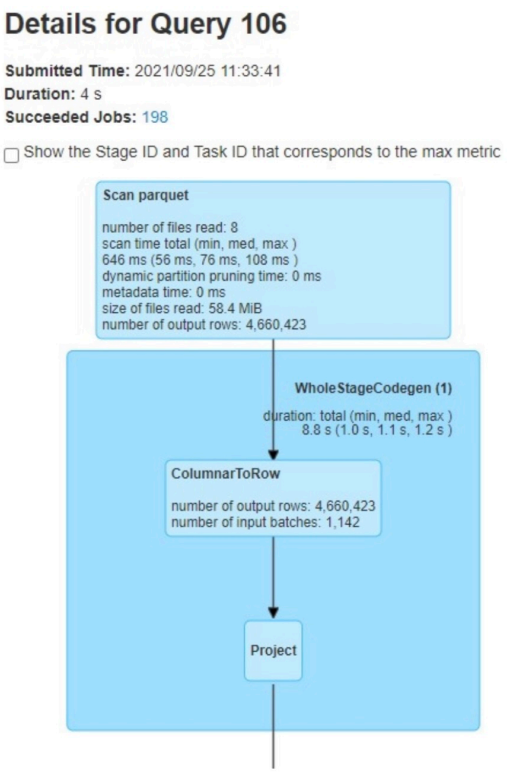


Figure 7: An example of the Spark UI SQL tab.

Debugging Using the Spark UI

It is quite easy to get lost within the Spark UI. It is best to spend some time familiarizing yourself with the Spark UI, the metrics that are available there, and most importantly, finding what you are looking for within the Spark UI. The best way to achieve this is by spending time tracing a single task through the Spark UI, in other words, launch a task in Spark and then trace it through the various tabs in the Spark UI. Also look at which stages are computationally the most expensive. As we have done within this train, you can create mock datasets that you can change to see how those changes impact performance. The Spark UI provides transparency into your application to a level that is not typically possible by simply analyzing the output of your application. It is important to combine the Spark UI insights with the output from `stdout` and `stderr`. Combining the puzzle pieces from all of the above sources will allow you a full picture.

Conclusion

In this lesson, we explored how to apply the SparkUI to monitor and optimise Spark applications. We learned how to navigate through key SparkUI tabs—including Jobs, Stages, Storage, Environment, Executors, and SQL—to identify performance bottlenecks, track memory and resource usage, and gain insights into task execution.

▼Key Takeaways ✖

- **Takeaway 1:** The SparkUI is a powerful interface for monitoring and optimising Spark applications.
- **Takeaway 2:** The Jobs and Stages tabs help identify execution timelines, bottlenecks, and metrics like task duration, garbage collection time, and shuffle data.
- **Takeaway 3:** The Storage tab shows cached or persisted tables and helps assess memory usage and partitioning.
- **Takeaway 4:** The Environment and Executors tabs reveal configuration details and executor performance, such as memory usage and active tasks.
- **Takeaway 5:** The SQL tab provides insights into executed SparkSQL queries, including row counts, shuffle data, and execution times.